

ECommerce Microservices Project - Complete Architecture Guide

Table of Contents

1. [Project Overview](#project-overview)
2. [Microservices Analysis](#microservices-analysis)
3. [Serilog + Elasticsearch + Kibana Guide](#serilog--elasticsearch--kibana-guide)
4. [Redis Cache Implementation](#redis-cache-implementation)
5. [CQRS + MediatR Pattern](#cqrs--mediatr-pattern)
6. [Polly and Resilience Patterns](#polly-and-resilience-patterns)
7. [Circuit Breaker Pattern](#circuit-breaker-pattern)

Project Overview

****Architecture****: Event-driven microservices with Clean Architecture pattern

****Backend****: .NET 10.0

****Frontend****: Angular 20/21

****Containerization****: Docker Compose

****API Gateway****: Ocelot

Microservices (6 Services)

Service	Port	Database	Architecture Layers
-----	-----	-----	-----
Catalog API	8000	MongoDB	API, Application, Core, Infrastructure
Basket API	8001	Redis	API, Application, Core, Infrastructure
Discount API	8002	PostgreSQL	API, Application, Core, Infrastructure
Ordering API	8003	SQL Server	API, Application, Core, Infrastructure
Payment API	8004	None (Event-driven)	Single layer
Identity API	8005	SQL Server	Single layer
API Gateway	8010	None	Single layer

Infrastructure Components

****Databases****:

- MongoDB (Catalog) - Port 27017
- Redis (Basket) - Port 6379
- PostgreSQL (Discount) - Port 5432
- SQL Server (Ordering, Identity) - Ports 1433, 1434

****Messaging & Logging**:**

- RabbitMQ with MassTransit - Ports 5672, 15672
- Elasticsearch + Kibana - Ports 9200, 5601
- Serilog for centralized logging

Architecture Patterns

****Clean Architecture**:** 4-layer separation (API, Application, Core, Infrastructure) for Catalog, Basket, Discount, Ordering services

****CQRS with MediatR**:** Command/Query pattern implemented across services

****Event-Driven Architecture**:**

- MassTransit over RabbitMQ
- Outbox pattern for reliable event publishing
- Retry policies (5 retries, 10-second intervals)

****Communication Patterns**:**

- ****Synchronous**:** HTTP/REST via Ocelot API Gateway
- ****gRPC**:** Basket → Discount (coupon validation)
- ****Async Events**:** Basket → Ordering → Payment flow

Event Flow

...

Basket Checkout → BasketCheckoutEvent → Ordering (Consumer)

↓

Order Created → OrderCreatedEvent → Payment (Consumer)

↓

Payment Result → PaymentCompletedEvent/PaymentFailedEvent →

Ordering

...

Microservices Analysis

Summary

Your ECommerce microservices project demonstrates solid architecture with Clean Architecture, CQRS, and event-driven patterns. The system includes 6 .NET 10.0

services (Catalog, Basket, Discount, Ordering, Payment, Identity) with an Angular 20 frontend, all containerized with Docker Compose.

Key Highlights

- Proper separation of concerns with 4-layer architecture
- Event-driven communication via RabbitMQ/MassTransit
- gRPC for high-performance Basket→Discount calls
- Centralized logging with ELK stack
- Ocelot API Gateway with JWT authentication

Areas for Improvement

1. **Identity Service**: JWT authentication code is commented out in ``Identity/Program.cs``
2. **Logging**: Serilog commented out in Ordering and Payment services
3. **HTTPS**: HTTPS redirection disabled in development for Basket, Ordering, Payment
4. **Rate Limiting**: Rate limiting configuration commented out in Ocelot
5. **Payment Service**: Minimal implementation with no database persistence
6. **No health checks**: Missing health check endpoints for services
7. **No distributed tracing**: Correlation ID middleware exists but no full tracing implementation

Serilog + Elasticsearch + Kibana Guide

Part 1: The Problem (Why We Need Logging)

Imagine you're running a lemonade stand with 6 friends helping you. Each friend has a different job:

- One makes the lemonade
- One takes orders
- One handles money
- One delivers to customers
- One manages coupons
- One handles customer accounts

Problem: When something goes wrong (like a customer says "I paid but didn't get my lemonade"), how do you figure out what happened? You can't watch everyone all the time!

****Solution****: You ask everyone to write down everything they do in a notebook:

- "At 2:30 PM, I took an order from John"
- "At 2:31 PM, I made lemonade for John"
- "At 2:32 PM, I delivered to John"

Now when there's a problem, you just read the notebooks to see what went wrong!

****In software****: Your microservices are like your friends. Serilog is the notebook system that helps them write down what they do.

Part 2: What is Serilog? (The Notebook Writer)

****Serilog**** is like a smart notebook system for your computer programs. Here's what it does:

****Without Serilog****:

```
```csharp
Console.WriteLine("Order created for John");
```
```

This just prints to the screen - once the program stops, the message is gone forever!

****With Serilog****:

```
```csharp
Log.Information("Order created for {UserName}", "John");
Log.Warning("Payment failed for order {OrderId}", 123);
Log.Error("Database connection lost!");
```
```

Serilog can:

- Save messages to multiple places (console, files, databases)
- Add extra information automatically (time, which service, environment)
- Format messages nicely
- Filter messages (show only important ones in production)

****Think of it like****: A smart assistant that writes down everything your programs do, and can save those notes in different places depending on what you need.

Part 3: What is Elasticsearch? (The Super-Fast Search Engine)

****Problem****: When you have thousands of notebooks (logs) from many services, finding a specific problem is like searching for a needle in a haystack!

****Elasticsearch**** is like a super-fast search engine for your logs. Think of it like Google, but for your application logs.

****What it does**:**

- Stores millions of log messages
- Can search through them in milliseconds
- Can find patterns (like "show me all errors in the last hour")
- Can count things (like "how many orders failed today?")

****Real-world example**:**

- Without Elasticsearch: You open 100 text files and search manually (takes hours)
- With Elasticsearch: You type "error" and it shows all errors in 0.1 seconds

****Think of it like**:** A magical library where you can find any book in seconds, even if there are millions of books!

Part 4: What is Kibana? (The Beautiful Dashboard)

****Problem**:** Elasticsearch is powerful but it's just text - like reading raw code. It's hard to see patterns and understand what's happening at a glance.

****Kibana**** is like a beautiful dashboard that shows your logs in charts, graphs, and tables. Think of it like the dashboard in your car, but for your application.

****What it shows you**:**

- ****Charts****: "How many errors per hour?" (line graph going up/down)
- ****Pie charts****: "Which service has the most errors?" (Catalog 30%, Basket 20%, etc.)
- ****Maps****: "Where are users coming from?" (world map with dots)
- ****Tables****: "Recent errors" (sorted list with details)
- ****Real-time****: Watch logs appear live as they happen

****Real-world example**:**

- Without Kibana: You see raw text like "ERROR at 2:30: Database failed"
- With Kibana: You see a red line graph showing error spike at 2:30, and can click to see details

****Think of it like**:** The difference between reading a spreadsheet of numbers vs. seeing colorful charts that tell a story!

Part 5: How They Work Together (The Complete Picture)

Think of it like a pizza delivery system:

```
...  
👨‍🍳 Your Microservice (Chef)  
  ↓ writes order  
📖 Serilog (Order Taker)  
  ↓ writes to notebook  
📦 Elasticsearch (Warehouse)  
  ↓ stores orders  
📊 Kibana (Manager Dashboard)  
  ↓ shows charts  
👤 You (Manager)  
  ↓ sees what's happening  
...
```

****Step-by-step flow**:**

1. ****Your code****: ``Log.Information("Order created for John")``
2. ****Serilog****: Grabs this message, adds time, service name, environment
3. ****Serilog****: Sends it to Elasticsearch (like sending to a super-fast database)
4. ****Elasticsearch****: Stores it and makes it searchable
5. ****Kibana****: Reads from Elasticsearch and shows beautiful charts

****Real example from your project**:**

- Catalog API logs: "Product added: Adidas Shoes"
- Serilog adds: Time=2:30 PM, Service=Catalog.API, Environment=Development
- Elasticsearch stores it with index: ``ecommerce-catalog-api-development-2025.04.29``
- Kibana shows you a chart: "10 products added today"

Part 6: Step-by-Step Implementation Guide

Step 1: Start the Infrastructure (The Foundation)

First, we need Elasticsearch and Kibana running. You already have them in your docker-compose!

```
```bash  
Run this command to start everything
docker-compose up -d elasticsearch kibana
```

```
```  
  
**What this does**: Starts Elasticsearch (port 9200) and Kibana (port 5601) in Docker containers
```

```
**Check if it's working**:
```

- Open browser: ``http://localhost:9200`` (should see JSON from Elasticsearch)
- Open browser: ``http://localhost:5601`` (should see Kibana login screen)

Step 2: Add Serilog to Your Service (The Setup)

You already have the ``Common.Logging`` project! Now let's use it in one of your services.

```
**Example: Enable Serilog in Ordering Service**
```

Open ``Ordering.API/Program.cs`` and find line 57 where Serilog is commented out:

```
```csharp  
// BEFORE (commented out):
//builder.Host.UseSerilog(Logging.ConfigureLogger);

// AFTER (uncomment it):
builder.Host.UseSerilog(Logging.ConfigureLogger);
```
```

```
**That's it!** Just one line enables Serilog.
```

Step 3: Add Configuration (Tell Serilog Where to Send Logs)

Open ``Ordering.API/appsettings.json`` and add this:

```
```json  
{
 "ElasticConfiguration": {
 "Uri": "http://elasticsearch:9200"
 }
}
```
```

```
**What this does**: Tells Serilog "Send logs to Elasticsearch at this address"
```

Step 4: Use Serilog in Your Code (Write Logs!)

Now you can log anywhere in your code:

```
```csharp
// In any controller or service
using Serilog;

public class OrderController
{
 public IActionResult CreateOrder(Order order)
 {
 Log.Information("Creating order for user {UserName}", order.UserName);

 try
 {
 // Your order logic here
 Log.Information("Order {OrderId} created successfully", order.Id);
 return Ok();
 }
 catch (Exception ex)
 {
 Log.Error(ex, "Failed to create order for user {UserName}",
order.UserName);
 return BadRequest();
 }
 }
}
```
```

Step 5: Run and Test (See It Work!)

```
```bash
Start your Ordering service
cd Services/Ordering/Ordering.API
dotnet run
```
```

****What happens**:**

1. Your service starts
2. Serilog automatically connects to Elasticsearch
3. Every log message gets sent to Elasticsearch

4. You see logs in console AND in Elasticsearch

Step 6: View Logs in Kibana (The Cool Part!)

1. Open `http://localhost:5601` in browser
2. Click "Stack Management" → "Index Patterns"
3. Click "Create index pattern"
4. Type: `ecommerce-*`` (this matches all your service logs)
5. Click "Next step" → select "@timestamp" → "Create index pattern"
6. Go to "Discover" to see your logs!

****You'll see**:**

- Time of each log
- Service name (Catalog.API, Basket.API, etc.)
- Log level (Information, Warning, Error)
- Your actual log message
- Any extra data you logged

Step 7: Enable in All Services (Complete Setup)

Repeat Steps 2-3 for all your services:

****Services to update**:**

- Catalog.API (already has Serilog enabled ✓)
- Basket.API (already has Serilog enabled ✓)
- Discount.API (needs Serilog added)
- Ordering.API (uncomment line 57)
- Payment.API (uncomment line 30)
- Identity.API (already has Serilog enabled ✓)

****For services without Serilog****, add to `Program.cs``:

```
```csharp
using Common.Logging;
using Serilog;

// Add this after var builder = WebApplication.CreateBuilder(args);
builder.Host.UseSerilog(Logging.ConfigureLogger);
```
```

And add to `appsettings.json``:

```

```json
{
 "ElasticConfiguration": {
 "Uri": "http://elasticsearch:9200"
 }
}
```

```

Step 8: Create a Dashboard (See the Magic!)

In Kibana:

1. Go to "Visualize Library"
2. Click "Create visualization"
3. Choose "Line chart"
4. Select your index pattern (``ecommerce-*``)
5. Choose "Count" for Y-axis
6. Choose "@timestamp" for X-axis (by hour)
7. Click "Save" → name it "Error Rate"

****Now you have a beautiful chart showing errors over time!****

Quick Reference Card

| Component | What it does | Where it runs |
|--------------------------|--------------------------|------------------------------|
| **Serilog** | Writes logs | Inside your .NET services |
| **Elasticsearch** | Stores and searches logs | Docker container (port 9200) |
| **Kibana** | Shows beautiful charts | Docker container (port 5601) |

Common Log Levels

```

```csharp
Log.Verbose("Super detailed info - only for debugging");
Log.Debug("Detailed info - development only");
Log.Information("Normal events - order created, user logged in");
Log.Warning("Something might be wrong - slow database");
Log.Error("Something went wrong - payment failed");
Log.Fatal("Critical - service is down!");
```

```

Your Current Setup (What You Have)

Looking at your `Logging.cs` file, you already have:

- ☒ Console logging (shows in terminal)
- ☒ Elasticsearch logging (sends to Elasticsearch)
- ☒ Automatic enrichment (adds service name, environment, time)
- ☒ Exception details (shows full error information)
- ☒ Per-service indexes (organized by service and date)

****You just need to**:**

1. Uncomment Serilog in Ordering and Payment services
2. Add Serilog to Discount service
3. Add Elasticsearch URL to all appsettings.json files

Redis Cache Implementation

Part 1: Why Redis Instead of Memory Cache? (The Problem)

Imagine you have a shopping cart on a website. You need to store what items the user added to their cart.

****Memory Cache (IMemoryCache)**** - Like Your Pocket:

- Stores data in your computer's RAM
- ****Problem****: If you restart your app, all data is gone!
- ****Problem****: If you have multiple servers (Server A and Server B), Server A's cache is different from Server B's cache
- ****Example****: User adds item on Server A, but Server B doesn't know about it

****Redis Cache**** - Like a Shared Notebook:

- Stores data in a separate server (Redis server)
- ****Benefit****: If you restart your app, data is still in Redis
- ****Benefit****: All servers share the same Redis - everyone sees the same data
- ****Benefit****: Super fast (in-memory storage)
- ****Benefit****: Can handle millions of requests per second

****Real-world example****:

- ****Memory Cache****: You write a note on your hand - if you wash your hands, note is gone
- ****Redis****: You write a note in a shared notebook - anyone can read it, and it stays there

****Why Basket Service Uses Redis****:

- Shopping carts are temporary (user might not checkout)
- Need speed (carts are accessed frequently)
- Multiple instances of Basket API might run (need shared cache)
- Don't want to lose carts if server restarts

Part 2: Required NuGet Packages

You need only **ONE** main package for Redis:

Package: `Microsoft.Extensions.Caching.StackExchangeRedis`` (Version 10.0.1)

What it does: Provides the interface to connect and work with Redis from .NET

Additional package (for JSON serialization):

Package: `Newtonsoft.Json`` (Version 13.0.4)

What it does: Converts C# objects to JSON strings (and back) so we can store them in Redis

Why Newtonsoft.Json?

- Redis stores data as strings
- Your ShoppingCart is a C# object
- Need to convert: C# Object → JSON String → Redis
- And when reading: Redis → JSON String → C# Object

How to install:

```
```bash
```

```
In your Infrastructure project
```

```
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

```
dotnet add package Newtonsoft.Json
```

```
```
```

Part 3: Step-by-Step Implementation Guide

Step 1: Start Redis Server (The Foundation)

You already have Redis in your docker-compose! Let's start it:

```
```bash
```

```
Start Redis
```

```
docker-compose up -d basketdb
```

```
Check if it's running
```

```
docker ps
```

```
```
```

****What this does****: Starts Redis container on port 6379

****Verify it's working****:

```
```bash
```

```
Connect to Redis CLI
```

```
docker exec -it basketdb redis-cli
```

```
Test with ping
```

```
PING
```

```
Should reply: PONG
```

```
Exit
```

```
exit
```

```
```
```

Step 2: Add NuGet Package to Your Project

In your ****Infrastructure**** project (Basket.Infrastructure):

```
```bash
```

```
cd Services/Basket/Basket.Infrastructure
```

```
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

```
dotnet add package Newtonsoft.Json
```

```
```
```

****Your .csproj file should look like this**** (you already have this!):

```
```xml
```

```
<ItemGroup>
```

```
 <PackageReference Include="Microsoft.Extensions.Caching.StackExchangeRedis"
Version="10.0.1" />
```

```
 <PackageReference Include="Newtonsoft.Json" Version="13.0.4" />
```

```
</ItemGroup>
```

```
```
```

Step 3: Configure Redis in Program.cs

In your ****API project**** (Basket.API/Program.cs), add Redis configuration:

```

```csharp
// Add this after line 45 (after GrpcSettings)

// Redis Configuration
builder.Services.AddStackExchangeRedisCache(options =>
{
 options.Configuration = builder.Configuration.GetSection("CacheSettings")
 .GetValue<string>("ConnectionString");
});
```

```

****What this does**:**

- Registers Redis as a distributed cache
- Reads connection string from configuration
- Makes `IDistributedCache` available for dependency injection

****Your current code (lines 66-70)**** already has this! ✓

Step 4: Add Connection String to appsettings.json

In ****Basket.API/appsettings.json****:

```

```json
{
 "CacheSettings": {
 "ConnectionString": "localhost:6379"
 }
}
```

```

****For Docker environment**** (docker-compose.override.yml), it uses:

```

```json
"CacheSettings__ConnectionString": "basketdb:6379"
```

```

****Why two different URLs?****

- `localhost:6379` - When running locally (your computer)
- `basketdb:6379` - When running in Docker (container name)

Step 5: Create Repository Interface

In ****Basket.Core/Repositories/IBasketRepository.cs****:

```
```csharp
using Basket.Core.Entities;

namespace Basket.Core.Repositories
{
 public interface IBasketRepository
 {
 Task<ShoppingCart> GetBasket(string userName);
 Task<ShoppingCart> UpsertBasket(ShoppingCart shoppingCart);
 Task DeleteBasket(string userName);
 }
}
```
```

****What this does****: Defines the contract for basket operations (you already have this! ✓)

Step 6: Implement Repository with Redis

In ****Basket.Infrastructure/Repositories/BasketRepository.cs****:

```
```csharp
using Basket.Core.Entities;
using Basket.Core.Repositories;
using Microsoft.Extensions.Caching.Distributed;
using Newtonsoft.Json;

namespace Basket.Infrastructure.Repositories
{
 public class BasketRepository : IBasketRepository
 {
 private readonly IDistributedCache _redisCache;

 public BasketRepository(IDistributedCache redisCache)
 {
 _redisCache = redisCache;
 }

 // GET cart from Redis
 public async Task<ShoppingCart> GetBasket(string userName)
 {
 // Implementation
 }
 }
}
```
```

```

    {
        var basket = await _redisCache.GetStringAsync(userName);
        if (string.IsNullOrEmpty(basket))
        {
            return null;
        }
        // Convert JSON back to C# object
        return JsonConvert.DeserializeObject<ShoppingCart>(basket);
    }

    // DELETE cart from Redis
    public async Task DeleteBasket(string userName)
    {
        await _redisCache.RemoveAsync(userName);
    }

    // SAVE cart to Redis
    public async Task<ShoppingCart> UpsertBasket(ShoppingCart shoppingCart)
    {
        // Convert C# object to JSON
        await _redisCache.SetStringAsync(
            shoppingCart.UserName,
            JsonConvert.SerializeObject(shoppingCart)
        );
        return await GetBasket(shoppingCart.UserName);
    }
}
}
...

```

****What each method does**:**

- ``GetBasket``: Reads JSON string from Redis → converts to ShoppingCart object
- ``UpsertBasket``: Converts ShoppingCart to JSON → saves to Redis
- ``DeleteBasket``: Removes the key from Redis

****Note**:** Your current code has ``RefreshAsync`` on line 33, but it should be ``RemoveAsync`` for deletion!

Step 7: Register Repository in DI Container

In **`**Basket.API/Program.cs**`** (line 26):

```
```csharp
```



```
builder.Services.AddScoped<IBasketRepository, BasketRepository>();
...
```

**\*\*What this does\*\*:** Makes `IBasketRepository` available for injection in controllers

#### #### Step 8: Use Repository in Controller

In your Basket Controller:

```
```csharp  
[HttpGet("{userName}")]  
public async Task<ActionResult<ShoppingCart>> GetBasket(string userName)  
{  
    var basket = await _basketRepository.GetBasket(userName);  
    return Ok(basket);  
}  
  
[HttpPost]  
public async Task<ActionResult<ShoppingCart>> UpdateBasket(ShoppingCart basket)  
{  
    var updatedBasket = await _basketRepository.UpsertBasket(basket);  
    return Ok(updatedBasket);  
}  
  
[HttpDelete("{userName}")]  
public async Task<IActionResult> DeleteBasket(string userName)  
{  
    await _basketRepository.DeleteBasket(userName);  
    return NoContent();  
}  
```
```

### ### Part 4: Redis Operations - Code Examples

#### #### Basic Redis Operations

**\*\*1. SET - Store data in Redis\*\***

```
```csharp  
// Store a simple string  
await _redisCache.SetStringAsync("user:123", "John Doe");
```

```

// Store with expiration (30 minutes)
await _redisCache.SetStringAsync(
    "user:123",
    "John Doe",
    new DistributedCacheEntryOptions
    {
        AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(30)
    }
);
```

2. GET - Retrieve data from Redis
```csharp
// Get a string
string userName = await _redisCache.GetStringAsync("user:123");
// Returns: "John Doe" or null if not found
```

3. DELETE - Remove data from Redis
```csharp
// Remove a key
await _redisCache.RemoveAsync("user:123");
```

4. Store Complex Objects (like ShoppingCart)
```csharp
// Convert object to JSON and store
var cart = new ShoppingCart
{
    UserName = "john",
    Items = new List<CartItem> { ... }
};

string jsonCart = JsonConvert.SerializeObject(cart);
await _redisCache.SetStringAsync($"basket:{cart.UserName}", jsonCart);
```

5. Retrieve Complex Objects
```csharp
// Get JSON and convert back to object
string jsonCart = await _redisCache.GetStringAsync($"basket:john");
var cart = JsonConvert.DeserializeObject<ShoppingCart>(jsonCart);
```

```

### ### Your Current Implementation Breakdown

Looking at your `BasketRepository.cs`:

#### **\*\*Line 22-28: GetBasket\*\***

```
```csharp
var basket = await _redisCache.GetStringAsync(userName); // Read from Redis
if (string.IsNullOrEmpty(basket))
    return null;
return JsonConvert.DeserializeObject<ShoppingCart>(basket); // JSON → Object
```
```

#### **\*\*Line 36-40: UpsertBasket\*\***

```
```csharp
await _redisCache.SetStringAsync(
    shoppingCart.UserName, // Key = username
    JsonConvert.SerializeObject(shoppingCart) // Value = JSON
);
```
```

#### **\*\*Key-Value Storage Pattern\*\*:**

- **\*\*Key\*\***: `userName` (e.g., "john", "rahul")
- **\*\*Value\*\***: JSON string of ShoppingCart
- **\*\*Example\*\***:
  - Key: ` "john" `
  - Value: `{ "UserName": "john", "Items": [ { "ProductId": "123", "Quantity": 2 } ] } `

### ### Advanced Redis Options

#### **\*\*Set Expiration (Auto-delete after time)\*\***

```
```csharp
await _redisCache.SetStringAsync(
    "basket:john",
    JsonConvert.SerializeObject(cart),
    new DistributedCacheEntryOptions
    {
        AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(24) // Delete after
24 hours
    }
);
```
```

```
Sliding Expiration (Reset timer on access)
```csharp
new DistributedCacheEntryOptions
{
    SlidingExpiration = TimeSpan.FromMinutes(30) // Reset timer every time
accessed
}
```
```

### ### Testing Your Redis Implementation

#### **\*\*1. Start Redis\*\***

```
```bash
docker-compose up -d basketdb
```
```

#### **\*\*2. Run your Basket API\*\***

```
```bash
cd Services/Basket/Basket.API
dotnet run
```
```

#### **\*\*3. Test with Postman/curl\*\***

```
```bash
# Create basket
POST http://localhost:8001/api/v1/Basket
{
  "userName": "john",
  "items": [
    {
      "quantity": 2,
      "price": 3500,
      "productId": "602d2149e773f2a3990b47f5"
    }
  ]
}

# Get basket
GET http://localhost:8001/api/v1/Basket/john

# Delete basket
DELETE http://localhost:8001/api/v1/Basket/john
```
```

#### **\*\*4. Check Redis directly\*\***

```
```bash
# Connect to Redis CLI
docker exec -it basketdb redis-cli
```

```
# List all keys
KEYS *
```

```
# Get specific key
GET "john"
```

```
# Exit
exit
```
```

#### **### Summary**

**\*\*Your Current Redis Implementation Status\*\*:** ☒ Already implemented correctly!

#### **\*\*What you have\*\*:**

- ☒ NuGet package: `Microsoft.Extensions.Caching.StackExchangeRedis` (10.0.1)
- ☒ NuGet package: `Newtonsoft.Json` (13.0.4)
- ☒ Redis configuration in Program.cs (lines 66-70)
- ☒ Connection string in appsettings.json
- ☒ Repository implementation with Redis
- ☒ Docker Compose with Redis container

**\*\*Bug Found\*\*:** In `BasketRepository.cs` line 33, you have:

```
```csharp
await _redisCache.RefreshAsync(userName); // WRONG - this refreshes expiration
```
```

#### **\*\*Should be\*\*:**

```
```csharp
await _redisCache.RemoveAsync(userName); // CORRECT - this deletes the key
```
```

#### **### Quick Reference**

| Component | Purpose |
|-----------|---------|
|-----------|---------|

```
|-----|-----|
| **Redis Server** | Fast in-memory key-value store (port 6379) |
| **StackExchangeRedis** | .NET library to talk to Redis |
| **IDistributedCache** | Interface for distributed caching |
| **Newtonsoft.Json** | Converts objects to/from JSON for storage |
```

### ### Why Redis for Shopping Carts?

1. **\*\*Speed\*\***: Millisecond response times
2. **\*\*Shared\*\***: Multiple API instances see same cart data
3. **\*\*Persistent\*\***: Survives server restarts
4. **\*\*Temporary\*\***: Perfect for carts that might not checkout
5. **\*\*Scalable\*\***: Handles millions of concurrent users

Your implementation follows best practices for distributed caching in microservices!

---

## ## CQRS + MediatR Pattern

### ### Part 1: The Problem CQRS Solves

Imagine you have a simple controller without CQRS:

```
```csharp
// WITHOUT CQRS - Everything in one place
public class BasketController : ControllerBase
{
    [HttpGet("{userName}")]
    public async Task<ActionResult> GetBasket(string userName)
    {
        // 1. Validation logic
        if (string.IsNullOrEmpty(userName))
            return BadRequest();

        // 2. Business logic
        var basket = await _repository.GetBasket(userName);

        // 3. Database logic
        // 4. Mapping logic
        // 5. Discount calculation
    }
}
```

```

        // 6. Event publishing
        // ALL MIXED TOGETHER!
    }
}
...

```

****Problems with this approach**:**

1. ****Controller becomes huge**** - Too much code in one place
2. ****Hard to test**** - Can't test business logic without HTTP
3. ****Hard to reuse**** - Can't use the same logic from another place
4. ****Violates Single Responsibility**** - Controller does too many things
5. ****Read and Write mixed**** - Reading data and writing data use same code path

****CQRS Solution****: Separate "Commands" (write operations) from "Queries" (read operations)

Part 2: Your Basket Project Structure

Your Basket project follows CQRS perfectly:

```

...
Basket.Application/
├── Commands/           ← WRITE operations (change data)
│   ├── CreateShoppingCartCommand.cs
│   ├── DeleteBasketByUserNameCommand.cs
│   └── BasketCheckoutCommand.cs
├── Queries/           ← READ operations (get data)
│   └── GetBasketByUserNameQuery.cs
├── Handlers/          ← The logic for each command/query
│   ├── CreateShoppingCartHandler.cs
│   ├── DeleteBasketByUserNameHandler.cs
│   ├── BasketCheckoutHandler.cs
│   └── GetBasketByUserNameHandler.cs
...

```

Part 3: Step-by-Step Implementation Guide

Step 1: Install MediatR Package

****Your file****: `Basket.Application/Basket.Application.csproj` (line 18)

```

...xml

```

```
<PackageReference Include="MediatR" Version="14.0.0" />
```
```

**\*\*What MediatR does\*\*:** It's a messenger that sends commands/queries to the right handler. Think of it like a post office - you give it a letter (command), and it delivers it to the right person (handler).

#### #### Step 2: Register MediatR in Program.cs

**\*\*Your file\*\*:** `Basket.API/Program.cs` (lines 32-39)

```
```csharp
// Register MediatR
var assemblies = new Assembly[]
{
    Assembly.GetExecutingAssembly(),
    typeof(CreateShoppingCartHandler).Assembly // This tells MediatR where
to find handlers
};
builder.Services.AddMediatR(cfg =>
cfg.RegisterServicesFromAssemblies(assemblies));
```
```

**\*\*What this does\*\*:**

- Scans your assemblies to find all handlers
- Automatically registers them with MediatR
- Now when you send a command, MediatR knows which handler to call

#### #### Step 3: Create a Query (READ Operation)

**\*\*Your file\*\*:** `Basket.Application/Queries/GetBasketByUserNameQuery.cs`

```
```csharp
using Basket.Application.Responses;
using MediatR;

namespace Basket.Application.Queries
{
    public record GetBasketByUserNameQuery(string UserName) :
    IRequest<ShoppingCartResponse>;
}
```
```



**\*\*Breakdown\*\*:**

- ``record`` - A C# feature for immutable data (like a class but simpler)
- ``GetBasketByUserNameQuery`` - The name of your query
- ``string UserName`` - The input data (what you need to search)
- ``: IRequest<ShoppingCartResponse>`` - Tells MediatR this is a query that returns ``ShoppingCartResponse``

**\*\*Think of it like\*\*:** A form you fill out - "I want to get basket for username 'john'"

#### #### Step 4: Create a Query Handler (The Logic)

**\*\*Your file\*\*:** ``Basket.Application/Handlers/GetBasketByUserNameHandler.cs``

```
```csharp
public class GetBasketByUserNameHandler :
    IRequestHandler<GetBasketByUserNameQuery, ShoppingCartResponse>
{
    private IBasketRepository _basketRepository;
    private ILogger<GetBasketByUserNameHandler> _logger;

    public GetBasketByUserNameHandler(IBasketRepository basketRepository,
    ILogger<GetBasketByUserNameHandler> logger)
    {
        _basketRepository = basketRepository;
        _logger = logger;
    }

    public async Task<ShoppingCartResponse> Handle(GetBasketByUserNameQuery
request, CancellationToken cancellationToken)
    {
        var shoppingCart = await _basketRepository.GetBasket(request.UserName);
        if(shoppingCart == null)
        {
            return new ShoppingCartResponse(request.UserName)
            {
                Items = new List<ShoppingCartItemResponse>()
            };
        }
        _logger.LogInformation("Fetched basket for
{@UserName}", request.UserName);
        return shoppingCart.ToShoppingCartResponse();
    }
}
```

```
    }  
}  
...
```

****Breakdown**:**

- `IRequestHandler<GetBasketByUserNameQuery, ShoppingCartResponse>` - Implements the handler interface
- Constructor - Gets dependencies (repository, logger) via dependency injection
- `Handle` method - The actual logic:
 1. Gets basket from Redis repository
 2. If not found, returns empty basket
 3. Logs the operation
 4. Converts entity to response DTO

****Think of it like**:** The worker who receives the form and does the actual work

Step 5: Create a Command (WRITE Operation)

****Your file**:** `Basket.Application/Commands/CreateShoppingCartCommand.cs`

```
```csharp  
using Basket.Application.DTOS;
using Basket.Application.Responses;
using MediatR;

namespace Basket.Application.Commands
{
 public record CreateShoppingCartCommand(string UserName,
List<CreateShoppingCartItemDto> items) : IRequest<ShoppingCartResponse>;
}
...
```

#### **\*\*Difference from Query\*\*:**

- Commands are for CHANGING data (create, update, delete)
- Queries are for READING data (get, search)
- Both look similar, but their purpose is different

#### **#### Step 6: Create a Command Handler (The Logic)**

**\*\*Your file\*\*:** `Basket.Application/Handlers/CreateShoppingCartHandler.cs`

```
```csharp
```

```

public class CreateShoppingCartHandler :
IRequestHandler<CreateShoppingCartCommand, ShoppingCartResponse>
{
    private readonly IBasketRepository _basketRepository;
    private readonly DiscountGrpcService _discountGrpcService;

    public CreateShoppingCartHandler(IBasketRepository basketRepository,
DiscountGrpcService discountGrpcService)
    {
        _basketRepository = basketRepository;
        _discountGrpcService = discountGrpcService;
    }

    public async Task<ShoppingCartResponse> Handle(CreateShoppingCartCommand
request, CancellationToken cancellationToken)
    {
        // Apply discounts using GRPC call
        foreach (var item in request.items)
        {
            var coupon = await
_discountGrpcService.GetDiscount(item.ProductName);
            item.Price = item.Price - coupon.Amount; // minus amount from
discount amount
        }
        // Convert command to domain entity
        var shoppingCartEntity = request.ToShoppingCartEntity();
        // Save to redis
        var updateCart = await
_basketRepository.UpsertBasket(shoppingCartEntity);
        // Convert back to response
        return updateCart.ToShoppingCartResponse();
    }
}
...

```

****What this handler does**:**

1. ****Business logic****: Calls gRPC service to get discounts
2. ****Calculation****: Subtracts discount from price
3. ****Conversion****: Converts command to domain entity
4. ****Persistence****: Saves to Redis
5. ****Response****: Converts back to response DTO

****Notice****: Complex business logic is in the handler, NOT in the controller!

Step 7: Use in Controller (The Entry Point)

****Your file**:** `Basket.API/Controllers/BasketController.cs`

```
```csharp
[HttpGet("{userName}")]
public async Task<ActionResult<ShoppingCartDto>> GetBasket(string userName)
{
 var query = new GetBasketByUserNameQuery(userName); // Create query
 var result = await _mediator.Send(query); // Send to MediatR
 _logger.LogInformation("Fetching Basket for {@userName}", userName);
 return Ok(result);
}

[HttpPost]
public async Task<ActionResult<ShoppingCartDto>> CreateOrUpdateBasket([FromBody]
CreateShoppingCartCommand createShoppingCartCommand)
{
 var result = await _mediator.Send(createShoppingCartCommand); // Send to
MediatR
 return Ok(result);
}
```
```

****What the controller does**:**

1. Creates the command/query object
2. Sends it to MediatR via `\_mediator.Send()`
3. MediatR finds the right handler
4. Handler executes the logic
5. Result returns to controller
6. Controller returns HTTP response

****Controller is now thin**** - just HTTP stuff, no business logic!

Part 4: The Complete Flow (Visual)

...

User Request



Controller (BasketController.cs)



Query/Command (GetBasketByUserNameQuery.cs)

```
    ↓ sends to  
MediatR (The Messenger)  
    ↓ finds  
Handler (GetBasketByUserNameHandler.cs)  
    ↓ calls  
Repository (BasketRepository.cs)  
    ↓ reads from  
Redis  
    ↓ returns  
Response  
```
```

### ### Part 5: Complex Example - Checkout Command

**\*\*Your file\*\*:** `Basket.Application/Commands/BasketCheckoutCommand.cs`

```
```csharp  
public record BasketCheckoutCommand(BasketCheckoutDto BasketCheckoutDto) :  
IRequest<Unit>;  
```
```

**\*\*Note\*\*:** Returns `Unit` (like `void`) because checkout doesn't return data - it just performs an action.

**\*\*Your file\*\*:** `Basket.Application/Handlers/BasketCheckoutHandler.cs`

```
```csharp  
public async Task<Unit> Handle(BasketCheckoutCommand request, CancellationToken  
cancellation_token)  
{  
    var basketDto = request.BasketCheckoutDto;  
  
    // 1. Get the basket (using MediatR to call another query!)  
    var basketResponse = await _mediator.Send(new  
GetBasketByUserNameQuery(basketDto.UserName), cancellation_token);  
  
    if (basketResponse is null || !basketResponse.Items.Any())  
    {  
        throw new InvalidOperationException("Basket not found or empty");  
    }  
  
    // 2. Convert to entity
```

```

    var basket = basketResponse.ToEntity();

    // 3. Map to event
    var evt = basketDto.ToBasketCheckoutEvent(basket);

    // 4. Publish event to RabbitMQ
    _logger.LogInformation("Publishing BasketCheckoutEvent For {User}",
basket.UserName);
    await _publishEndpoint.Publish(evt, cancellationToken);

    // 5. Delete the basket (using MediatR to call another command!)
    await _mediator.Send(new DeleteBasketByUserNameCommand(basket.UserName),
cancellationToken);

    return Unit.Value;
}
...

```

****This shows the power of CQRS + MediatR**:**

- Handler can call other handlers via MediatR
- Clean separation of concerns
- Each handler does one thing well
- Easy to test each piece independently

Part 6: Benefits You Get

****1. Clean Controllers****

- Before: Controller with 100+ lines of logic
- After: Controller with 3-5 lines (just calls MediatR)

****2. Easy Testing****

- Test handlers without HTTP
- Mock dependencies easily
- Unit test business logic in isolation

****3. Reusability****

- Same handler can be called from:
 - HTTP API
 - gRPC service
 - Background worker
 - Another handler

****4. Separation of Concerns****

- Controller: HTTP concerns
- Handler: Business logic
- Repository: Data access

****5. Scalability****

- Can optimize reads and writes separately
- Can use different databases for read/write (advanced)

Part 7: Summary

****CQRS**** = Command Query Responsibility Segregation

- ****Commands****: Change data (Create, Update, Delete)
- ****Queries****: Read data (Get, Search)

****MediatR**** = The messenger

- Sends commands/queries to handlers
- Automatically finds the right handler
- Handles dependency injection

****Your Implementation**:**

- ☒ Commands in ``Commands/`` folder
- ☒ Queries in ``Queries/`` folder
- ☒ Handlers in ``Handlers/`` folder
- ☒ Clean controllers that just call MediatR
- ☒ Business logic in handlers
- ☒ Easy to test and maintain

This is professional-grade architecture used in enterprise applications!

Polly and Resilience Patterns

Part 1: The Problem Polly Solves

Imagine you're calling a friend on the phone:

****Without Polly**:**

```

You call friend → No answer → You give up immediately

Result: You never get through!

```

****With Polly**:**

```

You call friend → No answer → Wait 10 seconds → Call again

→ No answer → Wait 10 seconds → Call again

→ No answer → Wait 10 seconds → Call again

→ Friend answers! Success!

```

****In Microservices**:**

- Services call each other over the network
- Network can fail (slow, timeout, connection lost)
- Databases can be temporarily unavailable
- External APIs can have issues

****Without Polly**:** One failure = entire request fails

****With Polly**:** Automatically retry and handle failures gracefully

Part 2: Where Polly is Used in Your Project

Location 1: Database Migration Retry (Ordering Service)

****File**:** `Ordering.API/Extensions/DbExtension.cs` (lines 21-28)

```
```csharp
// retry strategy
var retry = Policy.Handle<SqlException>()
 .WaitAndRetry(
 retryCount: 5,
 sleepDurationProvider: retryAttempt => TimeSpan.FromSeconds(Math.Pow(2,
retryAttempt)),
 onRetry: (exception, span, count) =>
 {
 logger.LogError($"Retrying because of {exception} {span}");
 });
retry.Execute(() => CallSeeder(seeder, context, services));
```
```

****What this does**:**

- `Policy.Handle<SqlException>()` - Only retry if SQL exception occurs
- `retryCount: 5` - Retry up to 5 times


```
- `(TimeSpan.FromSeconds(Math.Pow(2, retryAttempt))` - Exponential backoff:  
- Retry 1: Wait 2 seconds (2^1)  
- Retry 2: Wait 4 seconds (2^2)  
- Retry 3: Wait 8 seconds (2^3)  
- Retry 4: Wait 16 seconds (2^4)  
- Retry 5: Wait 32 seconds (2^5)  
- `onRetry` - Log each retry attempt  
- `retry.Execute()` - Run the actual operation with retry logic
```

****Why this is needed**:** When the application starts, SQL Server might not be ready yet (especially in Docker). This retries the migration instead of failing immediately.

Location 2: MassTransit Retry (Ordering & Payment Services)

****File**:** `Ordering.API/Program.cs` (lines 36, 40, 45, 50)

```
```csharp  
cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
```
```

****File**:** `Payment/Program.cs` (line 20)

```
```csharp  
cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
```
```

****What this does**:**

- MassTransit uses Polly internally for retry
- `Interval(5, TimeSpan.FromSeconds(10))` - Retry 5 times, wait 10 seconds between each retry
- Applied to RabbitMQ message consumption

****Why this is needed**:** If RabbitMQ is temporarily unavailable or message processing fails, it will retry instead of losing the message.

Part 3: Resilience Patterns Step-by-Step

Pattern 1: Retry Pattern

****Problem**:** Temporary failures (network glitch, database restart)

****Solution****: Automatically retry the operation

****Your code example**** (DbExtension.cs):

```
```csharp
var retry = Policy.Handle<SqlException>()
 .WaitAndRetry(
 retryCount: 5,
 sleepDurationProvider: retryAttempt => TimeSpan.FromSeconds(Math.Pow(2,
retryAttempt))
);
```
```

****How it works****:

```
```
Attempt 1: Fails → Wait 2 seconds
Attempt 2: Fails → Wait 4 seconds
Attempt 3: Fails → Wait 8 seconds
Attempt 4: Fails → Wait 16 seconds
Attempt 5: Fails → Give up (throw exception)
```
```

****Exponential Backoff****: Each retry waits longer than the previous one. This prevents overwhelming the system when it's already struggling.

Pattern 2: Circuit Breaker Pattern

****Problem****: If a service is down, retrying won't help - it just wastes time

****Solution****: Stop trying for a while, then try again later

****Visual****:

```
```
Closed (Normal) → Open (Stop trying) → Half-Open (Test) → Closed (Normal)
```
```

****Example**** (NOT in your code, but this is how it would look):

```
```csharp
var circuitBreaker = Policy
 .Handle<Exception>()
 .CircuitBreaker(
 exceptionsAllowedBeforeBreaking: 3,
 durationOfBreak: TimeSpan.FromSeconds(30)
);
```
```

```
```\n\n**How it works**:
```

- After 3 failures, stop trying for 30 seconds
- After 30 seconds, try once (Half-Open)
- If success, go back to normal (Closed)
- If failure, stay open for another 30 seconds

**\*\*Your project doesn't use Circuit Breaker yet\*\*** - you could add it for external service calls!

#### #### Pattern 3: Timeout Pattern

**\*\*Problem\*\***: Operation hangs forever (slow database, network timeout)

**\*\*Solution\*\***: Give up after a specific time

**\*\*Example\*\*** (NOT in your code):

```
```csharp\nvar timeout = Policy\n    .Timeout(TimeSpan.FromSeconds(30));\n```\n
```

****How it works****:

- If operation takes more than 30 seconds, throw TimeoutException
- Prevents your application from hanging

Pattern 4: Fallback Pattern

****Problem****: Operation fails, but you need to return something

****Solution****: Return a default value or cached data

****Example**** (NOT in your code):

```
```csharp\nvar fallback = Policy\n    .Handle<Exception>()\n    .Fallback("Default Value");\n```\n
```

**\*\*How it works\*\***:

- If operation fails, return "Default Value" instead of throwing exception

- Good for: returning cached data, default response, or empty list

### ### Part 4: Polly Package Installation

**\*\*Your file\*\***: `Ordering.API/Ordering.API.csproj` (line 21)

```
```xml
<PackageReference Include="Polly" Version="8.6.5" />
```
```

**\*\*To install\*\***:

```
```bash
dotnet add package Polly
```
```

### ### Part 5: Complete Example - Adding Polly to Basket Service

Let's say you want to add retry to the Discount gRPC call in Basket service:

**\*\*Without Polly\*\*** (current code in CreateShoppingCartHandler.cs):

```
```csharp
var coupon = await _discountGrpcService.GetDiscount(item.ProductName);
```
```

**\*\*With Polly\*\*** (how to add it):

```
```csharp
using Polly;

// In Program.cs - register Polly
builder.Services.AddHttpClient();

// In Handler - use Polly
var retryPolicy = Policy
    .Handle<Grpc.Core.RpcException>()
    .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(2));

var coupon = await retryPolicy.ExecuteAsync(async () =>
{
    return await _discountGrpcService.GetDiscount(item.ProductName);
});
```
```

### ### Part 6: MassTransit Retry (Your Current Implementation)

**\*\*Your code\*\*** (Ordering.API/Program.cs lines 36, 40, 45, 50):

```
```csharp
cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
```
```

**\*\*This is Polly retry under the hood!\*\*** MassTransit uses Polly for its retry mechanism.

**\*\*What it does\*\*:**

- When consuming messages from RabbitMQ
- If processing fails, retry up to 5 times
- Wait 10 seconds between each retry
- After 5 failures, move message to dead-letter queue

**\*\*Why this is critical\*\*:**

- Messages shouldn't be lost
- Temporary failures (database timeout, network glitch) shouldn't cause message loss
- After retries, message goes to dead-letter queue for manual inspection

### ### Part 7: Summary of Your Polly Usage

| Location                           | Pattern                        | Purpose                             |
|------------------------------------|--------------------------------|-------------------------------------|
| ----- ----- -----                  |                                |                                     |
| <b>**DbExtension.cs**</b>          | Retry with Exponential Backoff | Database migration retry on startup |
| <b>**Ordering.API/Program.cs**</b> | Retry (via MassTransit)        | RabbitMQ message consumption retry  |
| <b>**Payment/Program.cs**</b>      | Retry (via MassTransit)        | RabbitMQ message consumption retry  |

### ### Part 8: Why You Need Polly in Microservices

**\*\*Microservices Reality\*\*:**

- Services call each other over network (unreliable)
- Databases can be temporarily unavailable
- External APIs can have rate limits or downtime
- Network glitches happen

#### **\*\*Without Polly\*\*:**

- One failure = entire request fails
- Bad user experience
- Lost messages
- Cascading failures (one service down brings down others)

#### **\*\*With Polly\*\*:**

- Automatic retry on temporary failures
- Circuit breaker prevents overwhelming struggling services
- Timeout prevents hanging
- Fallback provides graceful degradation
- Better user experience
- System stays resilient

---

## **## Circuit Breaker Pattern**

### **### Part 1: Circuit Breaker Status in Your Project**

**\*\*Answer: You are NOT using Circuit Breaker in your project yet.\*\***

Your current Polly usage:

- ☒ **\*\*Retry Pattern\*\*** - Used in ``DbExtension.cs`` and MassTransit
- ☐ **\*\*Circuit Breaker Pattern\*\*** - NOT implemented anywhere

### **### Part 2: What is Circuit Breaker? (The Concept)**

Imagine a light switch in your house:

```

Switch OFF (Circuit Open) → No electricity flows

Switch ON (Circuit Closed) → Electricity flows normally

```

#### **\*\*In Software\*\*:**

- **\*\*Closed (Normal)\*\***: Everything works, requests go through
- **\*\*Open (Stop)\*\***: Too many failures, stop trying for a while
- **\*\*Half-Open (Test)\*\***: Try one request to see if service recovered

#### **\*\*Real-world example\*\*:**

```

You call a friend who is busy:

Call 1: No answer

Call 2: No answer

Call 3: No answer

→ You stop calling for 30 minutes (Circuit Open)

→ After 30 minutes, you try once (Half-Open)

→ If they answer: Go back to normal calling (Closed)

→ If no answer: Wait another 30 minutes (Stay Open)

...

****Why this is better than just retrying**:**

- ****Retry****: Keep trying even if service is down (wastes time/resources)
- ****Circuit Breaker****: Stop trying when service is clearly down (saves time/resources)

Part 3: Where to Implement Circuit Breaker in Your Project

****Best place****: Basket service's gRPC call to Discount service

****Current code**** (`Basket.Application/Handlers/CreateShoppingCartHandler.cs` lines 24-28):

```
```csharp
foreach (var item in request.items)
{
 var coupon = await _discountGrpcService.GetDiscount(item.ProductName);
 item.Price = item.Price - coupon.Amount;
}
```
```

****Problem****: If Discount service is down, this will keep failing and retrying (if you have retry), wasting time and resources.

****Solution****: Add Circuit Breaker to stop trying after a few failures.

Part 4: Step-by-Step Implementation Guide

Step 1: Add Polly NuGet Package (if not already there)

****File****: `Basket.Application/Basket.Application.csproj`

```
```xml
<ItemGroup>
```

```

 <PackageReference Include="Polly" Version="8.6.5" />
</ItemGroup>
```

**Install command**:
```bash
cd Services/Basket/Basket.Application
dotnet add package Polly
```

#### Step 2: Create a Policy Service (Best Practice)

Create a new file: `Basket.Application/Services/DiscountPolicy.cs`

```csharp
using Polly;
using Grpc.Core;

namespace Basket.Application.Services
{
 public static class DiscountPolicy
 {
 public static IAsyncPolicy<Grpc.Core.RpcException>
GetCircuitBreakerPolicy()
 {
 return Policy
 .Handle<RpcException>() // Handle gRPC exceptions
 .CircuitBreakerAsync(
 exceptionsAllowedBeforeBreaking: 3, // Break after 3
failures
 durationOfBreak: TimeSpan.FromSeconds(30) // Stay open for
30 seconds
);
 }

 public static IAsyncPolicy<Grpc.Core.RpcException> GetRetryPolicy()
 {
 return Policy
 .Handle<RpcException>()
 .WaitAndRetryAsync(
 retryCount: 3,
 sleepDurationProvider: retryAttempt =>
TimeSpan.FromSeconds(2)

```



```

 });
 }
}
```

#### Step 3: Combine Retry + Circuit Breaker

Update the policy service:

```csharp
using Polly;
using Grpc.Core;

namespace Basket.Application.Services
{
 public static class DiscountPolicy
 {
 public static IAsyncPolicy<Grpc.Core.RpcException> GetDiscountPolicy()
 {
 // Retry first, then Circuit Breaker
 return Policy
 .Handle<RpcException>()
 .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(2))
 .WrapAsync(
 Policy
 .Handle<RpcException>()
 .CircuitBreakerAsync(
 exceptionsAllowedBeforeBreaking: 3,
 durationOfBreak: TimeSpan.FromSeconds(30)
)
);
 }
 }
}
```

```

****What this does**:**

- First: Retry up to 3 times with 2-second delay
- Then: If still failing after retries, open circuit for 30 seconds
- Wrap: Combines both policies (Retry inside Circuit Breaker)

Step 4: Use in Handler

****File**:** `Basket.Application/Handlers/CreateShoppingCartHandler.cs`

```
````csharp
using Basket.Application.Commands;
using Basket.Application.GrpcService;
using Basket.Application.Mappers;
using Basket.Application.Responses;
using Basket.Application.Services; // Add this
using Basket.Core.Repositories;
using MediatR;
using Polly; // Add this

namespace Basket.Application.Handlers
{
 public class CreateShoppingCartHandler :
 IRequestHandler<CreateShoppingCartCommand, ShoppingCartResponse>
 {
 private readonly IBasketRepository _basketRepository;
 private readonly DiscountGrpcService _discountGrpcService;
 private readonly IAsyncPolicy<RpcException> _discountPolicy; // Add this

 public CreateShoppingCartHandler(
 IBasketRepository basketRepository,
 DiscountGrpcService discountGrpcService)
 {
 _basketRepository = basketRepository;
 _discountGrpcService = discountGrpcService;
 _discountPolicy = DiscountPolicy.GetDiscountPolicy(); // Initialize
policy
 }

 public async Task<ShoppingCartResponse> Handle(CreateShoppingCartCommand
request, CancellationToken cancellationToken)
 {
 // Apply discounts using GRPC call with Circuit Breaker
 foreach (var item in request.items)
 {
 var coupon = await _discountPolicy.ExecuteAsync(async () =>
 {
 return await
_discountGrpcService.GetDiscount(item.ProductName);
 });
 }
 }
 }
}
```

```

 item.Price = item.Price - coupon.Amount;
 }

 // Convert command to domain entity
 var shoppingCartEntity = request.ToShoppingCartEntity();

 // Save to redis
 var updateCart = await
_basketRepository.UpsertBasket(shoppingCartEntity);

 // Convert back to response
 return updateCart.ToShoppingCartResponse();
}
}
}
...

```

#### #### Step 5: Add Fallback (Optional but Recommended)

When circuit is open, return default discount (0):

```

```csharp
public static IAsyncPolicy<Grpc.Core.RpcException>
GetDiscountPolicyWithFallback()
{
    var circuitBreaker = Policy
        .Handle<RpcException>()
        .CircuitBreakerAsync(
            exceptionsAllowedBeforeBreaking: 3,
            durationOfBreak: TimeSpan.FromSeconds(30)
        );

    var fallback = Policy
        .Handle<RpcException>()
        .FallbackAsync(async (exception) =>
        {
            // Return default discount when circuit is open
            return new Discount.Grpc.Protos.CouponModel
            {
                Amount = 0,
                ProductName = "default",
                Description = "Discount service unavailable"
            };
        });
}

```

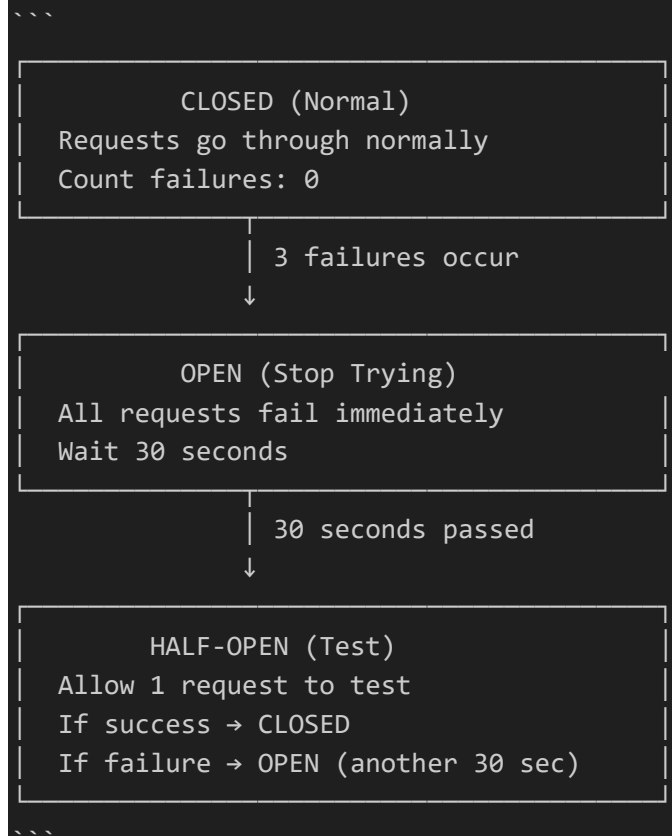
```

    });

    return fallback.WrapAsync(circuitBreaker);
}
```

```

#### #### Step 6: Circuit Breaker States Explained



#### #### Step 7: Monitor Circuit Breaker State (Advanced)

Add logging to track state changes:

```

```csharp
public static IAsyncPolicy<Grpc.Core.RpcException> GetCircuitBreakerWithLogging()
{
    return Policy
        .Handle<RpcException>()
        .CircuitBreakerAsync(
            exceptionsAllowedBeforeBreaking: 3,
            durationOfBreak: TimeSpan.FromSeconds(30),

```

```

        onBreak: (exception, breakDelay) =>
        {
            Console.WriteLine($"Circuit OPEN due to: {exception.Message}.
Will stay open for {breakDelay.TotalSeconds} seconds");
        },
        onReset: () =>
        {
            Console.WriteLine("Circuit CLOSED - service recovered");
        },
        onHalfOpen: () =>
        {
            Console.WriteLine("Circuit HALF-OPEN - testing service");
        }
    );
}
...

```

Part 5: Complete Working Example

****File**:** `Basket.Application/Services/DiscountPolicy.cs`

```

```csharp
using Polly;
using Grpc.Core;

namespace Basket.Application.Services
{
 public static class DiscountPolicy
 {
 public static IAsyncPolicy<Grpc.Core.RpcException> GetDiscountPolicy()
 {
 // Retry policy: Try 3 times with 2-second delay
 var retryPolicy = Policy
 .Handle<RpcException>()
 .WaitAndRetryAsync(
 retryCount: 3,
 sleepDurationProvider: retryAttempt =>
 TimeSpan.FromSeconds(2),
 onRetry: (exception, timeSpan, retryCount, context) =>
 {
 Console.WriteLine($"Retry {retryCount} after
{timeSpan.TotalSeconds}s due to: {exception.Message}");
 }
);
 }
 }
}

```

```

);

 // Circuit Breaker: Open after 3 failures, stay open for 30 seconds
 var circuitBreakerPolicy = Policy
 .Handle<RpcException>()
 .CircuitBreakerAsync(
 exceptionsAllowedBeforeBreaking: 3,
 durationOfBreak: TimeSpan.FromSeconds(30),
 onBreak: (exception, breakDelay) =>
 {
 Console.WriteLine($"Circuit OPEN for
{breakDelay.TotalSeconds}s due to: {exception.Message}");
 },
 onReset: () =>
 {
 Console.WriteLine("Circuit CLOSED - service recovered");
 },
 onHalfOpen: () =>
 {
 Console.WriteLine("Circuit HALF-OPEN - testing service");
 }
);

 // Combine: Retry first, then Circuit Breaker
 return Policy.WrapAsync(retryPolicy, circuitBreakerPolicy);
}
}
}
...

File: `Basket.Application/Handlers/CreateShoppingCartHandler.cs` (updated)

```csharp
public async Task<ShoppingCartResponse> Handle(CreateShoppingCartCommand request,
CancellationToken cancellationToken)
{
    var policy = DiscountPolicy.GetDiscountPolicy();

    foreach (var item in request.items)
    {
        try
        {
            var coupon = await policy.ExecuteAsync(async () =>

```

```

        {
            return await _discountGrpcService.GetDiscount(item.ProductName);
        });
        item.Price = item.Price - coupon.Amount;
    }
    catch (BrokenCircuitException)
    {
        // Circuit is open - use default discount (0)
        Console.WriteLine("Circuit is OPEN - using default discount");
        item.Price = item.Price; // No discount
    }
    catch (RpcException ex)
    {
        // Other gRPC errors
        Console.WriteLine($"gRPC error: {ex.Message}");
        item.Price = item.Price; // No discount
    }
}

var shoppingCartEntity = request.ToShoppingCartEntity();
var updateCart = await _basketRepository.UpsertBasket(shoppingCartEntity);
return updateCart.ToShoppingCartResponse();
}
...

```

Summary

****Circuit Breaker Status**:** NOT implemented in your project yet.

****Your Current Polly Usage**:**

- ☒ Retry Pattern (in `DbExtension.cs` and MassTransit)
- ☐ Circuit Breaker Pattern (not implemented)

****Best Place to Add Circuit Breaker**:** Basket service's gRPC call to Discount service (`CreateShoppingCartHandler.cs`)

****Implementation Steps**:**

1. Add Polly package to `Basket.Application`
2. Create `DiscountPolicy.cs` with Circuit Breaker + Retry
3. Update `CreateShoppingCartHandler.cs` to use the policy
4. Handle `BrokenCircuitException` when circuit is open

****Benefits**:**

- Stops calling Discount service when it's clearly down
- Saves time and resources
- Prevents cascading failures
- Better user experience (uses default discount instead of failing)

Conclusion

This comprehensive guide covers the key architectural patterns and technologies used in your ECommerce microservices project:

1. ****Project Overview****: 6 microservices with Clean Architecture, event-driven communication, and containerization
2. ****Serilog + Elasticsearch + Kibana****: Centralized logging with powerful search and visualization
3. ****Redis Cache****: Distributed caching for shopping carts with high performance
4. ****CQRS + MediatR****: Separation of read/write operations with clean architecture
5. ****Polly Resilience Patterns****: Retry patterns for handling temporary failures
6. ****Circuit Breaker Pattern****: (Not yet implemented) Prevents cascading failures

Your project demonstrates professional-grade microservices architecture with proper separation of concerns, event-driven communication, and resilience patterns. The implementation follows best practices for scalable and maintainable enterprise applications.